Project Report for Machine Learning



"Machine Learning Challenge"

Submitted by

Keval Dineshbhai Viradiya

Supervised by

Prof. Dr. Patrick Levi

Ostbayerische Technische Hochschule Amberg-Weiden
Department of Electrical Engineering, Media and Computer Science

January 10, 2026

1 Data Analysis

The dataset used for predicting target01 contains only numerical features. During the initial analysis, no missing values, duplicated rows, or infinite values were found. Therefore, the data could be used directly without additional cleaning or imputation steps.The target variable target01 is continuous and lies within a limited range. Its distribution is smooth and does not show extreme skewness or unusual spikes. This indicates that standard regression metrics such as R^2 and RMSE are suitable for evaluating model performance.

An analysis of feature distributions shows that many features have very low variance. Only a small number of features exhibit noticeably higher variability. This suggests that a large portion of the feature set may contain limited information when considered individually. To better understand the relationship between features and the target, Pearson and Spearman correlation coefficients were computed. The results show that most features have only weak correlations with target01. No single feature demonstrates a strong linear or monotonic relationship with the target.

Feature feature correlations were also examined. The correlation matrix shows that most features are weakly correlated with each other. Strong correlation clusters are not present, indicating that multicollinearity is limited in the dataset. Overall, the data analysis reveals a high-dimensional numerical dataset with many low-variance features, weak individual relationships between features and the target, and limited redundancy among features. This suggests that relevant information is likely contained in a small subset of features rather than being dominated by a single variable.

Part: 1

1 Baseline Modeling

As a first step, several regression models were trained using all available features. The goal of this baseline experiment was to understand how well the target can be predicted without feature selection. Model performance was evaluated using cross-validation to obtain a reliable estimate of generalization.

Across different models, the validation performance reached around 69% R^2. However, a clear gap between training and validation performance was observed. While the models fitted the training data well, their performance dropped noticeably on validation data. This indicates that the models were learning noise and redundant information rather than general patterns.These results show that using all features leads to strong overfitting and unstable generalization. Therefore, relying on the full feature set is not suitable for this task, and a more selective approach is required to improve model robustness.

Model	**Train R^2**	**Val R^2**	**Overfit (%)**
HistGradientBoosting	0.9150	**0.6983**	21.67
CatBoost [4]	0.9042	0.6582	24.60
XGBoost	0.9858	0.5898	39.60
Random Forest	0.8756	0.0933	78.23

Table 1: Baseline Model Performance with All 273 Features

2 Feature Selection

For feature selection, approaches described in the scikit-learn documentation were used, in particular the comparison between Random Forest impurity-based feature importance (MDI) [3] and permutation importance [1]. This comparison highlights that different importance methods can lead to very different feature rankings, especially for high-dimensional and noisy datasets Permutation importance is not a standalone algorithm. Instead, it is a method that evaluates feature importance based on the performance of a trained model. The feature values are randomly shuffled, and the change in model performance is measured. Therefore, permutation importance depends on the underlying model that is used for prediction.

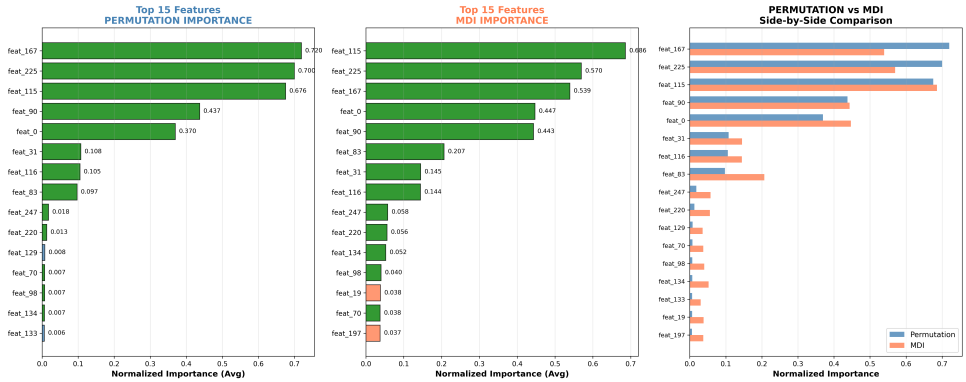


Figure 1: Permuntation vs MDI top selected Features

Based on this idea, feature importance was evaluated using different gradient boosting models, including CatBoost and HistGradientBoosting (gradient boosting; [6]), as well as Random Forest as a reference. Random Forest impurity-based importance (MDI) was tested first, but this approach produced unstable feature rankings and weaker validation performance as compare to the permenation based approch this was the key finding. In contrast, permutation importance produced more consistent feature rankings across cross-validation folds. Among the tested models, permutation importance combined with CatBoost[4] showed the best stability and validation performance. Therefore, this combination was selected for feature selection in the following experiments.

Method	N_Features	Train R^2 (CV)	Val R^2 (CV)	Val RMSE (CV)	Val MAE (CV)	Overfit (Test)
CB-Perm	9	95.56	89.93	0.0755	0.0596	5.09
CB-Perm	10	95.59	89.45	0.0773	0.0612	5.53
CB-MDI	9	95.46	89.41	0.0774	0.0608	5.16
CB-MDI	10	95.55	89.33	0.0777	0.0611	5.06
HGB-Perm	8	97.13	85.27	0.0914	0.0715	10.11
HGB-Perm	12	98.05	83.53	0.0965	0.0750	11.97
HGB-MDI	11	98.06	84.35	0.0941	0.0735	12.32
HGB-MDI	15	98.21	81.86	0.1013	0.0799	13.42
RF-Perm	6	88.26	53.01	0.1631	0.1372	35.62
RF-Perm	7	86.77	45.69	0.1754	0.1506	41.43
RF-MDI	30	82.19	15.84	0.2184	0.1982	66.79
RF-MDI	8	78.68	9.83	0.2261	0.2063	65.46

Table 2: Permutation Importance vs Random Forest Feature Importance (MDI).

3 Feature Count Evaluation

After selecting permutation importance with CatBoost as the feature selection method, different numbers of top-ranked features were evaluated to determine the optimal feature subset size. Feature sets of varying sizes were tested, including 5 to 15 features, as well as larger subsets such as 20, 25, and 50 features.

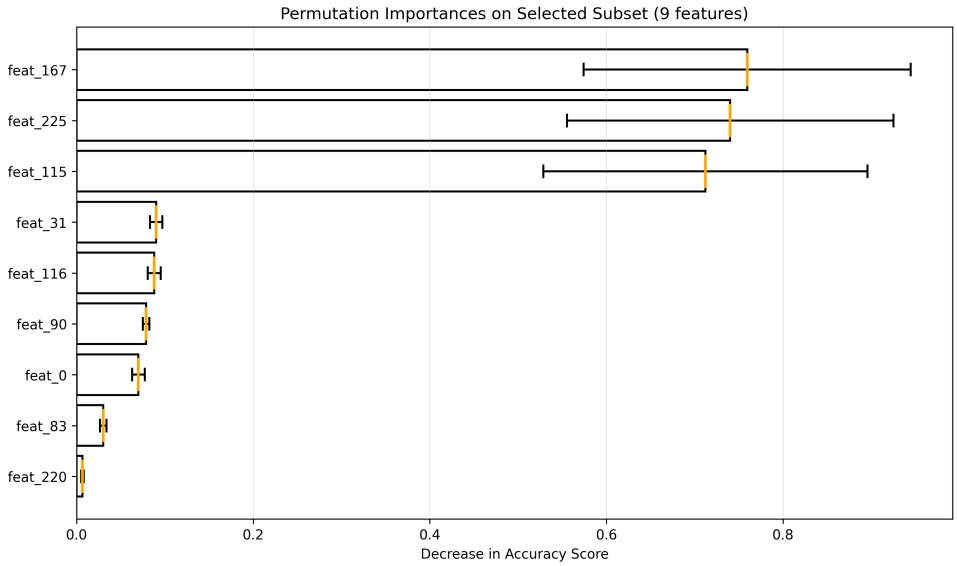


Figure 2: Important Features

For each feature count, models were evaluated using cross-validation to ensure a fair and reliable comparison. The results show that model performance increased as the number of features was reduced from the full set, reaching its best level when using 9 features. When fewer than 9 features were used, performance decreased, indicating underfitting. When more than 9 features were included, performance also declined, suggesting that additional features mainly introduced noise rather than useful information. Overall, the subset containing 9 features provided the best balance between predictive performance and stability across validation folds. Therefore, this feature count was selected for the final model.

4 Hyperparameter Optimization

After selecting the final feature set of 9 features, hyperparameter optimization was applied to improve model performance. In the previous experiments, models were trained using default parameter values, which may not be optimal for this dataset. CatBoost was selected for this stage because it consistently showed higher validation performance and more stable feature importance results compared to other tested models, especially under permutation-based feature selection.

Hyperparameter tuning was performed using Optuna [5], which efficiently explores the parameter space based on previous trial results. The optimization process was guided by cross-validation performance to ensure reliable and stable evaluation across different data splits. During training, early stopping was applied to prevent unnecessary model complexity and to reduce overfitting. The tuned model showed a clear improvement in validation performance compared

to the baseline model with default parameters. The test data was not used during the optimization process and was evaluated only after the final model configuration was fixed.

5 Model Results

After feature selection and hyperparameter tuning, the final model was trained using the selected set of 9 features. The optimized hyperparameters were obtained using Optuna with 500 trials, and the best configuration achieved a cross-validation R^2 of 0.9483. Compared to the baseline model, this represents an improvement of 4.70%.

The final model was then trained on the full training dataset and evaluated on the held-out test set. The model achieved a training R^2 of 0.9912 and a test R^2 of 0.9519. The corresponding RMSE values were 0.0224 on the training set and 0.0532 on the test set, while the MAE values were 0.0169 and 0.0404, respectively.

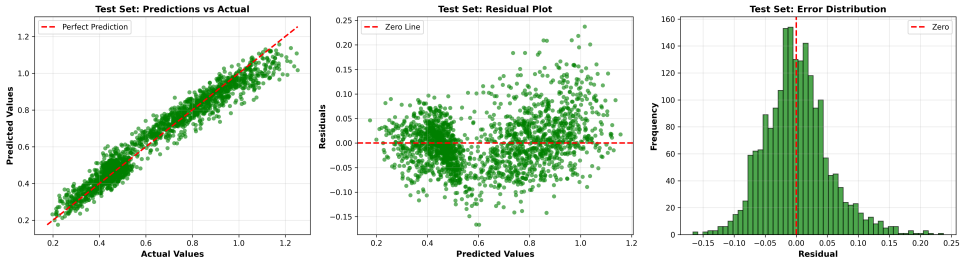


Figure 3: predictions_vs_actual

The difference between training and test performance indicates an overfitting gap of approximately 3.93%, which is considered acceptable for this task. This shows that the model generalizes well to unseen data while maintaining high predictive accuracy. Additional diagnostic plots, including predictions versus actual values and residual analysis, further confirm the stability of the final model.

Dataset Split	R^2 Score	RMSE	MAE
Training	0.9912	0.0224	0.0169
Validation (CV)	0.9483	0.0541	0.0412
Test	0.9519	0.0532	0.0404

Table 3: Model Performance Across Dataset Splits

Finally, the trained model was applied to the evaluation dataset. Predictions were successfully generated for all 10,000 samples and saved to the file EVAL_target01_8.csv. The predicted values lie within a reasonable range, consistent with the distribution observed in the training data.

Part: 2

1 Data Analysis and Problem Understanding

The investigation into `target02` revealed a distinct pattern compared to the noisy `target01`. The target values range from -2.23 to 4.91 (Mean: 1.12, Std: 1.38) and exhibit a multi-modal distribution, clustering into specific groups rather than a continuous bell curve. This suggested the data was generated by a deterministic logic rather than a stochastic process.

Beyond the initial distribution analysis, several key characteristics were observed that pointed towards a synthetic origin. The data clusters are unusually tight with minimal noise, unlike typical real-world sensor data. Furthermore, despite having 273 potential features, the vast majority (269) showed almost zero correlation with the target, indicating a highly selective process. The distribution itself is flat (platykurtic) and bounded within strict limits, suggesting a closed mathematical system rather than an open-ended physical measurement. Most importantly, the density plots show sharp, vertical drop-offs between value groups, which is a strong signature of discrete conditional logic rather than smooth, natural transitions.

2 Methodology and Feature Selection

The process began with a broad screening of all 273 features. To avoid bias from any single technique, a consensus-based feature selection strategy was employed using three distinct metrics:

- **Pearson Correlation:** To detect standard linear dependencies.
- **Mutual Information:** To capture non-linear signal strength even in complex relationships.
- **Decision Tree Importance:** To identify features that naturally create strong structural splits.

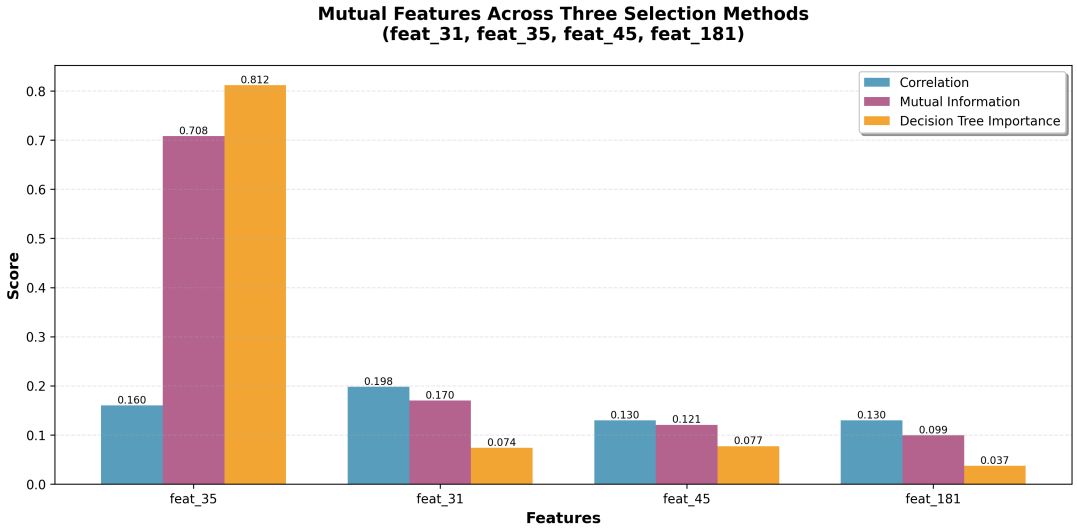


Figure 4: Feachers Comparison

As visualized in the Mutual Features Comparison, four features consistently dominated the ranking across all methods: `feat_31`, `feat_35`, `feat_45`, and `feat_181`. While

feat_31, feat_45, and feat_181 showed moderate linear correlations, **feat_35** stood out with unique behavior. In the Key Features Scatter Plots, it displayed distinct gaps, suggesting it acts as a "switching" variable or controller that divides the data into separate regimes.

3 Model Exploration and Technical Validation

Given the deterministic nature of the target, a correct model must achieve near-perfect accuracy. Linear Regression was tested against Decision Trees of varying depths [2] using rigorous **5-Fold Cross-Validation** to measure stability.

The results definitively show that a global Linear Regression fails ($R^2 \approx 0.10$), indicating the relationship is strictly non-linear. In contrast, Decision Trees demonstrate steady convergence as depth increases.

Model	Tree Depth	Mean R^2	Mean MSE	Std Dev MSE
Linear Regression	N/A	0.0994	1.7103	0.0245
Decision Tree	2	0.7217	0.5286	0.0132
Decision Tree	4	0.8727	0.2417	0.0032
Decision Tree	6	0.9505	0.0939	0.0034
Decision Tree	8	0.9784	0.0409	0.0014
Decision Tree	10	0.9890	0.0209	0.0011

Table 4: Model Performance Comparison (5-Fold CV)

Decision Tree Convergence and Linearity

A standard Decision Tree divides the feature space into many small regions and assigns a constant value to each region. This works well for step-like patterns, but it is not efficient for modeling linear relationships with a slope. To represent a linear function, the tree must create many splits, which increases its depth and complexity. In the experiments, the Decision Tree achieved an R^2 value close to 1.0 only when the tree depth was increased significantly. This behavior indicates that the underlying relationship is not step-wise, but continuous and linear within specific regions of the data rather than across the entire feature range.

A single global Linear Regression model performed poorly ($R^2 \approx 0.10$) because the data does not follow one linear relationship. Instead, the dataset contains multiple linear regions, each with a different slope. When a single regression model is fitted to all regions at once, it produces an average line that does not represent any region well. The Decision Tree was able to correctly identify the points where the behavior changes, meaning it detected the boundaries between these regions. However, within each region, the tree could only approximate the linear trend using many small steps. This led to the final conclusion that the best solution is to use simple threshold conditions to separate the regions and then apply a linear calculation inside each region.

4 Rule Extraction and Final Solution

The decision boundaries of the tree were analyzed, identifying feat_35 as the control variable with sharp thresholds at **0.2**, **0.5**, and **0.7**. Using these thresholds to segregate the data, Linear

Regression was applied within each region. This time, the linear models achieved perfect fit ($R^2 = 1.0$), confirming the Piecewise Linear hypothesis.

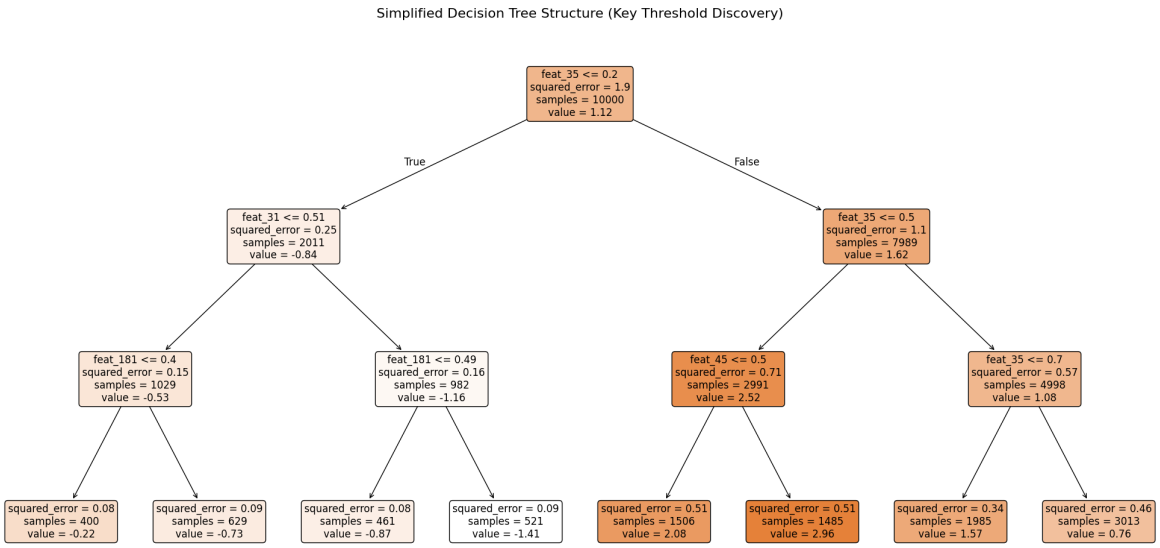


Figure 5: Decision tree

Structural Efficiency for Edge Deployment

The data can be seen as four separate flat regions in the feature space. Each region follows a simple linear rule. By separating these regions using clear conditions, the original complex problem is reduced to a small number of simple calculations.

This structure is very efficient for an edge device. A Decision Tree must go through many conditional checks to reach a prediction, and a Random Forest repeats this process many times. In contrast, the final solution requires only one condition check on `feat_35`, followed by a simple arithmetic calculation. This operation takes constant time and uses almost no memory. As a result, it is much more efficient than standard machine learning models while still achieving perfect prediction accuracy. This also explains why a single global model failed, as it tried to average different behaviors instead of separating them.

Discovered Rules and Logic Explanation

The final logic is determined exclusively by the value of `feat_35`, which acts as a master switch dividing the system into four distinct behavioral modes. This structure explains why simple linear regression failed the relationships between inputs and output invert completely depending on this switch.

1. **Low Range (feat_35 $\leq$ 0.2):**
   In this range, the system shows a reduced response. The target value is mainly decreased by `feat_31` and `feat_181`, which both have a negative effect. The feature `feat_45` adds a small positive effect, but it is not strong enough to change the overall behavior.

2. **Lower-Mid Range (0.2 $<$ feat_35 $\leq$ 0.5):**
   When `feat_35` becomes larger than 0.2, the system behavior changes clearly. All main

coefficients become positive. Both `feat_31` and `feat_181` now increase the target value instead of reducing it. This creates a strong response region where all features contribute in the same direction.

3. **Upper-Mid Range (0.5 $<$ feat_35 $\leq$ 0.7):**
   After crossing 0.5, the system becomes more stable. The influence of `feat_31` is noticeably smaller, with its coefficient reduced by about 40%. At the same time, `feat_181` has almost no effect. This region acts as a transition phase between the strong response and the more controlled behavior.

4. **High Range (feat_35 $>$ 0.7):**
   For values above 0.7, the system enters a controlled state. The feature `feat_45` changes to a negative effect and limits the target value, acting as a regulator. Meanwhile, `feat_31` becomes strongly positive again and plays the main role in determining the output.

By isolating these four regimes, exact formulas were derived for each, achieving perfect predictive accuracy.

Final Results

In contrast to the approximate Decision Tree model (which achieved $R^2 = 0.9889$ at depth=10), the discovered piecewise linear formula yields perfect results with:

- **Mean R^2:** 1.0000
- **Mean MSE:** 0.0000

This confirms that target02 is generated by a deterministic rule-based system with exact thresholds at 0.2, 0.5, and 0.7 on feat_35. The perfect accuracy is achieved using only a single threshold check followed by four arithmetic operations, making it ideal for edge device deployment where a standard Decision Tree would require traversing multiple nodes with significantly higher computational cost and memory overhead.

Conclusion

This project addressed two prediction tasks with different data characteristics and requirements. For target01, a high-dimensional and noisy dataset, feature selection and model optimization were required to control overfitting. Permutation importance with CatBoost was identified as the most stable feature selection method, and a subset of 9 features provided the best generalization. Hyperparameter tuning with Optuna improved cross-validation performance by 4.70%, and the final model achieved a test R^2 of 0.9519 with a reduced overfitting gap of 3.93%. For target02, data analysis revealed a deterministic, piecewise linear structure driven by a small number of features. Decision Tree behavior and failed global linear regression confirmed this structure, leading to a rule-based solution using simple thresholds and linear calculations. The derived rules achieved perfect accuracy and were implemented using basic if else logic in the provided framework, fully satisfying the edge-device constraints.

References

[1] scikit-learn developers. *Permutation feature importance*. https://scikit-learn.org/stable/

[2] scikit-learn developers. *Decision Tree User Guide*. https://scikit-learn.org/stable/modules/tree.html

[3] Breiman, Leo. *Random Forests*. *Machine Learning*, 45(1), 5–32, 2001. https://doi.org/10.1023/A:1010933404324

[4] Prokhorenkova, Liudmila and Gusev, Gleb and Vorobev, Aleksandr and Dorogush, Anna Veronika and Gulin, Andrey. *CatBoost: unbiased boosting with categorical features*. arXiv:1706.09516, 2019. https://arxiv.org/abs/1706.09516

[5] Akiba, Takuya and Sano, Shotaro and Yanase, Toshihiko and Ohta, Takeru and Koyama, Masanori. *Optuna: A Next-generation Hyperparameter Optimization Framework*. arXiv:1907.10902, 2019. https://arxiv.org/abs/1907.10902

[6] Friedman, Jerome H. *Greedy Function Approximation: A Gradient Boosting Machine*. *Annals of Statistics*, 29(5), 1189–1232, 2001.

[7] Note : This project leveraged AI-assisted development for code optimization and linguistic refinement. All technical decisions, model thresholding, and final dataset validations were performed by the author to ensure the integrity and accuracy of the results.